REINFORCEMENT LEARNING

# Policy Gradient Method

## Complete Mathematical Derivation

*From objective function to the final policy gradient expression*

## Overview

The **Vanilla Policy Gradient (REINFORCE)** is the fundamental algorithm for policy-based reinforcement learning. While it is rarely used directly in realistic settings, it is *critical for building intuition* and theoretical background for more complex algorithms like PPO, A3C, and TRPO.

| Approach | Description |
|---|---|
| Value-Based Methods | **Q-learning, DQN** — learn good action values first, then derive a policy from them |
| Policy-Based Methods | **Policy Gradient** — directly optimise the policy parameters θ with respect to the RL objective |

In policy gradient methods, when we **parameterise a policy** we will explicitly model π. Learning is now done by approximate **gradient ascent updates** on J(θ), where J is the expected return objective.

---

## Part 1 — The Objective Function J(θ)

### 1.1 Definition

We define the objective function as the **expected cumulative discounted reward** when following policy π$\theta$:

EQUATION 1 — OBJECTIVE FUNCTION

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t R(s_t, a_t)\right]$$

$$= \mathbb{E}_{\tau \sim \pi_\theta}\left[R(\tau)\right]$$

### 1.2 Terms Explained

| Symbol | Meaning |
|---|---|
| θ | Parameters of the policy (weights of the neural network) |
| π_θ | Parameterised policy — maps state s to probability distribution over actions |
| τ | Trajectory: the sequence $(s_0, a_0, r_0)$, $(s_1, a_1, r_1)$, ..., (sT,aT,rT) the agent observes. It is random with probability distribution P_θ, which is a function of θ |
| $R(s_t, a_t)$ | Reward function — unknown to the agent. Gives reward based on environment dynamics given state $s_t$ and action $a_t$ |
| R(τ) | Cumulative return from a trajectory — sum of discounted rewards |
| γ | Discount factor (0 < γ ≤ 1) — future rewards matter less than immediate ones |

> **Key Insight — What J(θ) Says**
>
> J(θ) is the expected total reward when following policy π$\theta$. We want to find θ that **maximises** J(θ). Since τ is a function of the policy, and the policy is a function of θ, J is ultimately a function of θ alone.

---

## Part 2 — Gradient Ascent on J(θ)

A natural approach to maximise J(θ) is to take a **gradient step in the ascending direction**. The update rule is:

> **PARAMETER UPDATE RULE**
>
> $$\theta_{k+1} = \theta_k + \alpha \cdot \nabla_\theta J(\pi_{\theta_k})$$

where α is the **step size (learning rate)**. The term $\nabla_\theta J(\pi\theta)$ is called the **policy gradient term** — and finding this term is the central challenge of the whole derivation.

> **The Core Problem**
>
> To compute $\nabla_\theta J(\theta)$, we need to know how reward changes as θ changes. But the **environment is a black box** — we cannot differentiate through it. We need a way to estimate this gradient purely from sampled trajectories.

---

# Part 3 — Expanding J(θ) Explicitly

We compute the **expected return J(θ) explicitly** by summing over all possible trajectories — weighted by the probability of taking each trajectory given θ:

**EQUATION 1 — EXPLICIT OBJECTIVE**

$$J(\theta) = \Sigma_{\tau} \; P(\tau; \theta) \cdot R(\tau) \qquad \ldots \text{ (Eq. 1)}$$

## 3.1 Expanding P(τ; θ)

The trajectory probability P(τ;θ) is a **chain of products** across every timestep — the probability of observing a state, taking a particular action given that state, and observing the next state:

**EQUATION 2 — TRAJECTORY PROBABILITY**

$$P(\tau; \theta) = \Pi_{t=0}^{T} \; P(s_{t+1} \mid s_t, a_t) \cdot \pi_\theta(a_t \mid s_t) \qquad \ldots \text{ (Eq. 2)}$$

$P(s_{t+1} \mid s_t, a_t)$: Environment dynamics (state transition)

$\pi_\theta(a_t \mid s_t)$: Policy probability (under our control)

| Term | Meaning |
|---|---|
| $P(s_{t+1} \mid s_t, a_t)$ | Environment state transition dynamics — probability of reaching next state. NOT controlled by θ |
| $\pi_\theta(a_t \mid s_t)$ | Policy probability — probability of taking action $a_t$ in state $s_t$ under parameters θ. This IS controlled by θ |

**Why This Matters**

Notice that P(τ;θ) contains **two kinds of terms**. Only $\pi\theta(a_t \mid s_t)$ depends on θ. The environment dynamics $P(s_{t+1} \mid s_t, a_t)$ do not. This observation is what allows us to eliminate the environment from the gradient computation in Part 5.

---

# Part 4 — Computing ∇_θ J(θ)

Taking the gradient of Equation 1 with respect to θ and moving it inside the sum:

$$\nabla_\theta J(\theta) = \nabla_\theta \Sigma_\tau P(\tau;\theta) \cdot R(\tau)$$

$$= \Sigma_\tau \nabla_\theta P(\tau;\theta) \cdot R(\tau)$$

The problem: $\nabla\theta P(\tau;\theta)$ is **impossible to compute directly** because it involves differentiating through the unknown environment dynamics. We need a trick.

## 4.1 The Log Derivative Trick

This trick (also called the **likelihood ratio trick**) is a simple rule of calculus:

LOG DERIVATIVE IDENTITY

$$\nabla_x \log f(x) = \nabla_x f(x) / f(x)$$

$$\text{Therefore:} \quad \nabla_x f(x) = f(x) \cdot \nabla_x \log f(x)$$

Applying this to our gradient — we multiply and divide by $P(\tau;\theta)$:

APPLYING THE LOG DERIVATIVE TRICK

$$\nabla_\theta J(\theta) = \Sigma_\tau \nabla_\theta P(\tau;\theta) \cdot R(\tau)$$

$$= \Sigma_\tau P(\tau;\theta) \cdot [\nabla_\theta P(\tau;\theta) / P(\tau;\theta)] \cdot R(\tau)$$

$$= \Sigma_\tau P(\tau;\theta) \cdot \nabla_\theta \log P(\tau;\theta) \cdot R(\tau) \quad \text{... (Eq. 2)}$$

$$= E_{\{\tau \sim \pi_\theta\}} [\nabla_\theta \log P(\tau;\theta) \cdot R(\tau)]$$

### Why This Is So Powerful

By applying the log derivative trick, we have converted **an impossible derivative** (through the environment) into **an expectation** that we can estimate by sampling! We simply run the policy, collect trajectories, and average.

This is the **Monte Carlo estimation** step — instead of computing the expectation exactly (which would require summing over all possible trajectories), we approximate it from m sampled trajectories:

$$\nabla_\theta J(\theta) \approx (1/m) \Sigma_i \nabla_\theta \log P(\tau^i ; \theta) \cdot R(\tau^i)$$

*By taking the average (1/m), we obtain an unbiased estimate that reduces the variance of the gradient estimate. More trajectories = less noisy gradient.*

---

# Part 5 — Simplifying ∇_θ log P(τ; θ)

We now expand $\nabla_\theta \log P(\tau;\theta)$. Using the full expansion of $P(\tau;\theta)$ from Equation 2:

**EXPANDING THE LOG**

$$\nabla_\theta \log P_\theta(\tau, \theta) = \nabla_\theta \log [\, \mu(s_0) \cdot \Pi_{t=0} P(s_{t+1}|s_t, a_t) \cdot \pi_\theta(a_t|s_t) \,]$$

Using the **log of a product = sum of logs** rule:

$$= \nabla_\theta \log \mu(s_0) + \Sigma_t \nabla_\theta \log P(s_{t+1}|s_t, a_t) + \Sigma_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$

## 5.1 Dropping the Environment Terms

Now we apply a crucial observation — two of the three terms have zero gradient with respect to θ:

| Term | Why It Drops Out |
|---|---|
| $\nabla_\theta \log \mu(s_0) = 0$ | **Initial state distribution**. The policy parameter θ does NOT affect which state the task starts in. Since $\mu(s_0)$ is independent of θ, its derivative with respect to θ is zero. |
| $\nabla_\theta \log P(s_{t+1}\|s_t, a_t) = 0$ | **State transition dynamics**. The policy parameters θ do not affect state-to-state transition probabilities (those are determined by the environment). So the derivative with respect to θ is zero. |

**Therefore**, the only term that survives is the policy term:

**SIMPLIFIED GRADIENT OF LOG TRAJECTORY PROBABILITY**

$$\nabla_\theta \log P(\tau;\theta) = \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

**The Environment Disappears Completely**

**This is the most elegant result of the derivation.** The environment dynamics — which are unknown and uncontrollable — vanish entirely from the gradient. Only the **policy log probability** remains, which we **can** compute and differentiate through our neural network.

---

## Part 6 — The Policy Gradient Expression

Substituting the simplified $\nabla_\theta \log P(\tau;\theta)$ back into Equation 2:

**FINAL — POLICY GRADIENT EXPRESSION**

$$\nabla_\theta J(\theta) = \sum_\tau P(\tau;\theta) \cdot \left[ \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \right] \cdot R(\tau)$$

$$= \mathbb{E}_{\tau \sim \pi_\theta} \left[ \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot R(\tau) \right]$$

This is the

### Policy Gradient Theorem.

In practice, $R(\tau)$ (the total trajectory return) is replaced by $\mathbf{G_t}$ — the discounted return from timestep t onwards. This gives the REINFORCE update rule:

**REINFORCE UPDATE RULE**

$$\theta \leftarrow \theta + \alpha \cdot \sum_t \left[ \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t \right]$$

$$\text{where} \quad G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots$$

---

## Part 7 — Complete Derivation at a Glance

**1** **Define objective**

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[ R(\tau) \right]$$

*Maximise expected return over trajectories sampled from the policy*

| | |
|---|---|
| 2 | **Write expectation as sum**<br>$$J(\theta) = \Sigma_\tau \; P(\tau;\theta) \cdot R(\tau) \qquad \text{(Eq. 1)}$$<br>*Sum over all possible trajectories, weighted by their probability* |
| 3 | **Take gradient**<br>$$\nabla_\theta J(\theta) = \Sigma_\tau \; \nabla_\theta P(\tau;\theta) \cdot R(\tau)$$<br>*Move gradient inside the sum* |
| 4 | **Apply log derivative trick**<br>$$= \Sigma_\tau \; P(\tau;\theta) \cdot \nabla_\theta \log P(\tau;\theta) \cdot R(\tau) \qquad \text{(Eq. 2)}$$<br>*Multiply and divide by P(τ;θ) → converts to an expectation we can sample* |
| 5 | **Expand log P(τ;θ)**<br>$$\nabla_\theta \log P(\tau;\theta) = \nabla_\theta \log \mu(s_0) + \Sigma_t \nabla_\theta \log P(s_{t+1}|s_t, a_t) + \Sigma_t \nabla_\theta \log \pi_\theta(a_t|s_t)$$<br>*Use log(product) = sum of logs* |
| 6 | **Drop zero-gradient terms**<br>$$\nabla_\theta \log P(\tau;\theta) = \Sigma_t \; \nabla_\theta \log \pi_\theta(a_t|s_t)$$<br>*$\mu(s_0)$ and $P(s_{t+1}|s_t, a_t)$ do not depend on $\theta$ → their gradients are zero* |
| 7 | **Substitute back into Eq. 2**<br>$$\nabla_\theta J(\theta) = E_{\tau}\left[ \Sigma_t \; \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot R(\tau) \right]$$<br>*Final policy gradient expression — environment has vanished completely* |
| 8 | **Estimate by sampling (Monte Carlo)**<br>$$\nabla_\theta J(\theta) \approx (1/m) \Sigma_i \; \Sigma_t \; \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$<br>*Run m episodes, use actual returns $G_t$ as estimates — this is REINFORCE* |

---

# Part 8 — Intuition Behind the Final Expression

## 8.1 What Each Term Does

| Term | Role |
|---|---|
| $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ | The score function. Points in the direction in θ-space that makes action $a_t$ more probable in state $s_t$. This is what we can compute by backpropagating through our neural network. |
| $G_t$ (or $R(\tau)$) | The return. Tells us how good this outcome was. Scales the gradient update — big return = big step, small return = small step, negative return = reverse direction. |
| · (multiply) | The core mechanism. High return → push hard toward making this action more likely. Low return → push gently. This is literally "reinforce what worked". |

| | |
|---|---|
| E_{τ} [ ... ] | The expectation. In practice estimated by averaging over m sampled episodes (Monte Carlo). More episodes = less noisy gradient estimate. |

### 8.2 Why the Environment Disappears

The key insight is that $\nabla\theta$ only acts on terms that depend on θ. The environment dynamics P(s'|s,a) and the initial state distribution $\mu(s_0)$ are **fixed by the world** — θ cannot change them. So their derivatives with respect to θ are zero and they drop out.

This means we can compute the policy gradient **without ever knowing the environment model**. We only need to be able to sample from it — which we can always do by running the agent.

### 8.3 Connection to LR-P (Learning Automata)

| Property | LR-P vs Policy Gradient |
|---|---|
| Policy representation | Probability vector p vs Neural network π_θ |
| Update on good outcome | p_i += a(1-p_i) vs θ += α · ∇log π · G |
| Update on bad outcome | p_i -= b·p_i vs θ += α · ∇log π · G (negative G) |
| Environment model | None (both) |
| States | No (LR-P) vs Yes (Policy Gradient) |

> **The Deep Connection**
>
> **LR-P is essentially a hand-coded, stateless, linear policy gradient algorithm.** Both say: *"actions that led to good outcomes should become more probable."* LR-P does it with arithmetic; REINFORCE does it with calculus and neural networks. The philosophy is identical.

---

## Part 9 — The REINFORCE Algorithm

Putting it all together, the complete REINFORCE algorithm is:

| | |
|---|---|
| 1 | **Initialise**<br>`Set θ randomly — start with an ignorant policy` |
| 2 | **Collect episode**<br>`Run π_θ → record (s0,a0,r0), (s1,a1,r1), ..., (sT,aT,rT)` |

*This is the Monte Carlo sampling step*

**3** **Compute returns**

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \text{ for each } t$$

*Walk backwards through episode — arithmetic only, not backprop*

**4** **Compute gradient**

$$\nabla_\theta J \approx \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot G_t$$

*Backpropagate through the neural network for $\nabla_\theta \log \pi_\theta$*

**5** **Update parameters**

$$\theta \leftarrow \theta + \alpha \cdot \nabla_\theta J$$

*Gradient ascent step*

**↺** **Repeat from Step 2**

**Until policy converges**

*Each repeat = one new Monte Carlo sample of the gradient*

### REINFORCE is Monte Carlo Policy Gradient

REINFORCE is called a Monte Carlo method because it estimates Q(s,a) — the true expected future return — by **actually playing the episode to the end** and using the observed return $G_t$ as the estimate. No model, no bootstrapping. Just raw sampled experience.

---

**Next:** *Variance problem & baseline subtraction → Actor-Critic → PPO*